



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

MPI 编程实验

姓名：吕昊远

学号：2410668

专业：计算机科学与技术

指导教师：王刚

2026 年 6 月 9 日

目录

一、 实验背景	1
二、 实验原理	1
(一) MPI 多进程并行	1
(二) 任务划分策略	2
(三) 通信与归并策略	2
(四) MPI 与多线程结合	3
(五) 图索引与 MPI	3
三、 程序设计与实现	4
(一) MPI 并行框架 (mpi_common.h)	4
(二) MPI-IVF: 任务划分与归并 (S0~S3)	4
(三) MPI 与多线程编程 (S4)	5
(四) MPI-IVF: 通信优化 (S5~S6)	5
(五) MPI-IVF-PQ (S7~S10)	6
(六) MPI-HNSW (S11~S14)	7
(七) 批量测试框架	8
四、 实验结果分析	8
(一) 数据总览	8
(二) 进程数扩展性分析	10
(三) 任务划分方式对比	11
(四) MPI 与多线程混合并行 ($P \times T$)	12
(五) local_topk 的影响	13
(六) 参数权衡分析	13
(七) 数据并行与任务并行对比	15
(八) 问题探讨	17
五、 总结与体会	17
六、 附录: 代码仓库链接	19

一、实验背景

在前两次的实验中，我们围绕近似最近邻搜索（ANNS）逐步展开优化从 SIMD 到多线程编程，在 IVF、IVF-PQ、HNSW 等进阶算法上探索了共享内存下的多核并行。然而，在前面的实验中，我们都局限在单进程的范畴内进行优化。

在本次实验中，我们在前两次的基礎上进一步引入 MPI (Message Passing Interface) **多进程并行**。多进程与多线程最本质的区别在于，MPI 的各个进程都拥有自己独立的地址空间与互不共享内存，进程间只能通过消息传递进行相关的通信。而这也使得并行优化的核心流程发生了转变。相比于直接在共享内存里给多个线程分配任务，我们还需考虑如何将数据或任务切分到各进程，并通过进程间的通信完成结果归并这一系列操作。因此，本实验的重心也主要在这几部分上，考虑任务划分、通信和归并的代价，以及如何将 MPI 与前面的 SIMD/多线程并行优化合理有效地结合起来。

依照实验要求，本次主要完成以下内容：

- 将之前已经实现的 IVF-SIMD 进行 MPI 并行优化，设计并对比不同的任务分配算法以及通信归并策略，分析不同算法的特点与性能差异。
- 探讨 MPI 与 OpenMP 的混合同行，分析进程数与线程数 ($P \times T$) 对性能与负载均衡的影响；
- 对 IVF-PQ 与 HNSW 也进行 MPI 编程优化，进一步探讨数据并行与任务并行两种策略在召回率与扩展性上的本质差异。

实验使用 DEEP100K 数据集，在实验提供的 ARM 架构服务器上运行与测试。本次实验以**单进程串行的 IVF - SIMD**（即 S0）作为全局基线，对上述内容进行系统性的实现、测试与分析。

二、实验原理

本实验在前两次实验的基础上进行，对于相关的算法，这里我们先作简单的回顾：

- **IVF（倒排文件索引）**：对基向量 KMeans 聚类得 `nlist` 个簇并内存重排；查询时先**粗排**选出最近的 `nprobe` 个簇，再**精排**扫描这些簇内向量。`nprobe` 是召回与延迟的核心权衡参数。
- **IVF-PQ**：在 IVF 粗筛后用 PQ 的 ADC 查表快速估算近似距离，筛出 `rerank_k` 个候选再用原始浮点精排。**方法一（全局 PQ）**全库共享一套量化器、LUT 仅构建一次；**方法二（簇内独立 PQ）**每簇独立训练量化器，精度更高但每个候选簇都需重建 LUT。
- **HNSW**：分层可导航小世界图，从顶层贪心导航逐层下降，底层做宽度为 `ef_search` 的集束搜索，查询复杂度对数级。本实验使用 `hnsplib` 库。

本次实验中，主要对于以上这几种方法进行 MPI 多进程编程，实现并分析相关的多进程并行策略，并继续探索在算法思路上的进一步优化。

（一）MPI 多进程并行

进程与线程的区别：通过前面的实验，我们知道线程是进行运算调度的最小单元，而本次实验涉及的进程，则可以看作操作系统资源分配的最小单元。对于同一进程内的多个线程，每个线程

间都会共享地址空间；而不同进程则拥有各自独立的地址空间，彼此的数据互不可见。因此，在 MPI 程序中，如果一个进程需要访问另一个进程的数据，就只能通过显式的消息传递来实现，使用点对点的 `Send/Recv` 或集合通信 `Gather/Reduce/Bcast` 等方式。

MPI 执行流程： 通过 `mpiexec -np P ./prog` 启动的程序会创建 P 个进程，这些进程会并行执行同一份代码。各进程通过 `MPI_Comm_rank` 获得自己的编号 `rank`，通过 `MPI_Comm_size` 获得本次运行的总进程数 P ，再依据自己的 `rank` 来决定负责的数据或任务。程序以 `MPI_Init` 开始、`MPI_Finalize` 结束，是这两者之间整体的并行，而非仅在程序中途某段并行。体现在运行的流程上，对应的数据加载等操作也会被各个进程重复执行 P 次。

加速比与扩展性： 沿用 $S_p = T_1/T_p$ 衡量加速比， T_1 为单进程基线延迟、 T_p 为 P 进程下的延迟。受 Amdahl 定律约束，串行部分占比 α 时加速上限为：

$$S_p \leq \frac{1}{\alpha + \frac{1-\alpha}{P}}$$

对于 MPI 而言，**进程间通信**构成了一类新的不可忽视的额外开销，而这也是本实验中进行扩展性分析的重点所在。

(二) 任务划分策略

将 N 个基向量分配到 P 个进程，主要有两种经典的划分方式：

- **块划分 (block)：** 将 $[0, N)$ 连续切成 P 段，进程 r 持有第 r 段。为保证负载均衡，前 $N \bmod P$ 个进程需要各多分一个向量。其特点是访存局部性较好，且实现简单。
- **循环划分 (cyclic)：** 将全局 id 为 i 的向量分配给进程 $i \bmod P$ 。其特点是可以将数据均匀地分配到各进程，在每个进程独立建索引、独立近似搜索的场景下，可以使各个分片的数据分布更加均衡。

由于各进程在自己的数据分片上都是独立构建索引的，其搜索返回的都是分片内的局部下标，因此必须通过一张局部下标与全局原始 id 的映射表将其还原为全局 id 之后，才能进行归并操作与结果比对，这是 MPI 实现过程中的一个关键细节。

(三) 通信与归并策略

各进程本地搜索得到局部 top- k 后，需要归并得到全局 top- k 。而在这一过程中，归并的方式直接决定了通信开销。为了探讨这部分的优化效果，本次实验主要实现并对比了四种策略：

- **Gather + root 线性归并：** 所有进程把局部 top- k 发送到 root，由 root 串行合并 $P \times k$ 个候选。实现简单，但当 P 较大时，root 会成为性能的瓶颈，而且对于每条 query，都需要进行一次集合通信，这也带来了许多不必要的开销。
- **树形 Reduce (自定义算子)：** 将“一条 query 的 top- k 列表”封装为自定义 MPI 数据类型，并定义一个可交换、可结合自定义规约算子，用于合并两份 top- k 列表。MPI 可以根据此以 $O(\log P)$ 的树形深度完成归并，且归并的负载会分摊到各层进程，缓解了 root 的串行瓶颈。
- **批量通信：** 将所有 query 先在本地全部搜索完，再用一次大规模的 Gather 统一传输、统一归并。这彻底消除了每次 query 的通信开销，是对于吞吐的关键优化。

- **非阻塞通信与流水线**: 利用非阻塞集合通信 `MPI_Igather` 配合双缓冲, 使当前 query 的本地计算与上一条 query 的结果通信相互重叠, 从而隐藏通信延迟, 使用流水线的思想进一步优化多进程。

(四) MPI 与多线程结合

从程序运行的角度来看, 进程与线程属于两层不同的概念。本次实验中, MPI 与多线程也可以分层组合来进行进一步的优化, 即进程间使用 MPI 并行、进程内使用多线程并行。具体实现上, 表现为进程间按数据分片并行、进程内对候选簇进行簇级的 OpenMP 动态调度。混合并行的关键在于 $P \times T$ 的配比: 受物理硬件的约束, $P \cdot T$ 不宜超过核数, 否则会因超额订阅导致严重的负优化; 同时还需要兼顾任务粒度, 当任务粒度过小时, 进程内 OpenMP 的线程调度开销很可能会抵消多线程带来的收益, 甚至出现负优化的情况。

本实验仅采用 OpenMP 与 MPI 相结合, Pthread 与 OpenMP 的并行逻辑基本一致, 在前一次的实验中已充分对比, 此处不再重复。

(五) 图索引与 MPI

对于图索引 (HNSW), 本次实验探索了三种不同的 MPI 并行方案。它们分属数据并行与任务并行两种范式, 二者在召回率与扩展性上具有一定的差异。

多分块 (数据并行): 将基向量按 block 划分为 P 个子集, 每个进程在自己的分片上独立构建一张 HNSW。查询时各进程在各自的子图上独立搜索, 再归并各进程的局部 top- k 。其本质是把数据切开、各自做近似搜索, 因此每个分片只是全局数据的一个子集, P 的变化会改变各子图的覆盖范围, 而这也意味着进程数 P 不仅影响速度, 也会改变最终的召回率。这与上次实验中 HNSW 多子图并行以广度换取召回精度原理类似: 单个子图使用较小的 `ef` 搜索路径较短, 但 P 个子图联合起来可以覆盖更大的空间范围, 从而在多线程下反而获得更高的召回率。在此基础上, 还可以进一步在进程内将分片再切分为多个子图、用 OpenMP 多线程并行搜索, 形成进程间 MPI、进程内多线程的多层并行。

IVF + HNSW (任务并行): 另一种思路是先对数据集构建统一的全局 IVF 索引, 即各进程持有相同的一份, 并为每个 IVF 簇单独建立一张 HNSW, 再将这些簇按编号分配给各个进程持有 (如簇 c 由进程 $c \bmod P$ 负责)。查询时, 各进程独立完成粗排选出 `nprobe` 个候选簇, 随后只搜索自己所持有的那部分候选簇的 HNSW, 最后归并。其核心特征在于无论进程数 P 取多少, 被搜索的候选簇集合始终相同, 进程间只是把按簇搜索这一任务拆分并行执行, 最终归并结果与单进程完全一致。因此, 这种方案下召回率与 P 严格无关, P 只影响搜索的速度, 这也正是任务并行区别于数据并行的本质所在。

HNSW-on-HNSW: 作为对 IVF+HNSW 的进一步探索, 我们还可以将其粗排环节也替换为图索引。具体来说, 就是在 `nlist` 个簇心上额外构建一张顶层路由 HNSW, 查询时先用顶层 HNSW 快速检索出最近的 `nprobe` 个簇, 再到这些簇对应的下层 HNSW 中精搜。这样便形成了上层图负责路由、下层图负责精搜的两层图结构, 其余的簇间 MPI 并行与归并逻辑则和 IVF + HNSW 一致。相比暴力扫描全部簇心的粗排, 当 `nlist` 规模较大时, 顶层 HNSW 的对数级时间复杂度可以显著降低粗排的开销; 而在 `nlist` 较小时, 二者差异则不是很明显。

三、 程序设计与实现

本实验复用了上次实验中大部分核心文件 (ivf.h、ivf_pq.h、ivf_pq_v2.h、hnswh.h)，在此基础上新增 MPI 多进程并行层。为了全面探讨各个算法的细节与特点，本实验一共实现了 S0~S14 共 15 个策略，按算法与并行方式分组如下：

- S0：无 MPI 的串行 IVF，作为全局基线 T_1
- S1~S6：MPI-IVF（块/循环划分、Gather/树形归并、批量通信、流水线、OpenMP 结合）
- S7~S10：MPI-IVF-PQ（方法一/方法二，纯 MPI 与 +OpenMP）
- S11~S14：HNSW 进阶（MPI-HNSW 多分块、MPI-IVF-HNSW、HNSW-on-HNSW）

接下来将逐步讲解各个算法及其 MPI 多进程优化的具体实现。

（一）MPI 并行框架 (mpi_common.h)

为了方便调用，首先将所有 MPI 策略所需的工具统一写为了一个头文件 mpi_common.h，包括任务划分、本地数据物化、全局 id 映射与 top- k 归并。其中任务划分按 block/cyclic 计算本进程负责的全局 id 列表；本地物化将分散的全局向量收集为连续缓冲，从而可以直接复用上次实验中已有的 build_ivf 等以连续基向量为输入的接口；全局 id 映射则在归并前把局部下标还原为全局原始 id。

Algorithm 1 任务划分与全局 id 映射 (mpi_common.h)

Input: 基向量总数 N , 进程号 $rank$, 进程总数 P , 划分方式 $part$

Output: 本进程负责的全局 id 列表 $shard_ids$

```

1: if  $part = \text{block}$  then
2:    $q \leftarrow \lfloor N/P \rfloor$ ,  $rem \leftarrow N \bmod P$ 
3:    $start \leftarrow rank \cdot q + \min(rank, rem)$ ;  $cnt \leftarrow q + [rank < rem]$ 
4:    $shard\_ids \leftarrow [start, start + 1, \dots, start + cnt - 1]$ 
5: else
6:    $shard\_ids \leftarrow \{i \mid i \bmod P = rank, 0 \leq i < N\}$ 
7: end if
8: return  $shard\_ids$ 

```

归并阶段: 各进程本地 top- k 队列中的下标 j 经 $shard_ids[j]$ 还原为全局 id

（二）MPI-IVF：任务划分与归并 (S0~S3)

S0 为无 MPI 的串行 IVF，在单进程上对全量数据构建 IVF 并搜索，作为后续 MPI 并行优化的基础以及全局加速比基准。该策略的实现与上次实验中完全一致，此处不再赘述。

S1 (block + Gather) 是 MPI-IVF 的基础版本，各进程在自己的数据分片上独立建立 IVF 索引，查询时各自取局部 top- k ，映射回全局 id 后通过两次 MPI_Gather（距离数组与 id 数组各一次）收集到 root，而后再由 root 归并出全局 top- k 。

Algorithm 2 MPI-IVF 块划分 + Gather 归并 (S1)

Input: 全量 base, 查询 q , k , nprobe, 进程号 $rank$, 进程总数 P

Output: 全局 top- k 结果（仅 root 完整）

```

1:                                     ▷ 离线: 各进程在分片 shard_ids 上独立 build_ivf
2:                                     ▷ 在线 (每条 query):
3: local_q ← ivf_search_simd(q, nprobe, k)                                     ▷ 局部 top-k, 局部下标
4: 将 local_q 转换为定长数组 (dist[], id[]), 并经 shard_ids 映射 id 为全局
5: MPI_Gather(dist) → rd[]; MPI_Gather(id) → rid[]                                     ▷ 收集到 root
6: if rank = 0 then
7:   result ← MergeTopK(rd, rid, k)                                     ▷ 归并  $P \times k$  个候选
8: end if
9: return result

```

S2 (cyclic + Gather) 则采用循环划分的策略。与 S1 相比, 两者仅在划分方式上有所不同, 主要用于对比划分策略对各个分片内的数据分布以及召回率的影响。

S3 (block + 树形 Reduce) 将归并方式由 Gather 改为树形 Reduce。其核心是构造一个自定义 MPI 数据类型。具体实现上, 就是把一条 query 的 top-*k* 列表 (*k* 个连续存储的 (*dist*, *id*) 对) 封装为一个 MPI 元素, 并定义一个用于合并两份 top-*k* 列表的自定义规约算子。该算子可交换、可结合, MPI 能够以对数深度的树形结构完成归并操作。

树形 Reduce 的自定义算子与数据类型 (mpi_common.h)

```

1 struct MpiCand { float d; uint32_t id; }; // 一个候选 (距离, 全局id) 对
2
3 // 自定义规约算子: inout = merge(in, inout), 保留距离最小的 K 个
4 void mpi_topk_merge_op(void* inv, void* inoutv, int* len, MPI_Datatype*) {
5     MpiCand* in = (MpiCand*)inv; MpiCand* io = (MpiCand*)inoutv;
6     // 将两份各 K 个候选合并, 用 max-heap 保留最优 K 个写回 io
7 }
8 // 把 K 个 MpiCand 连续封装为一个 MPI 数据类型, 并注册可交换的规约算子
9 MPI_Type_contiguous(K, cand_type, &topk_type); MPI_Type_commit(&topk_type);
10 MPI_Op_create(&mpi_topk_merge_op, /*commute=*/1, &topk_op);
11
12 // 查询时可以直接调用 MPI_Reduce(send, recv, 1, topk_type, topk_op, 0,
    MPI_COMM_WORLD);

```

(三) MPI 与多线程编程 (S4)

S4 在 S1 (block + Gather) 的基础上, 将进程内的本地搜索由单线程改为 OpenMP 簇级动态并行 (即调用多线程实验中的 ivf_search_omp)。这样形成进程间 MPI + 进程内 OpenMP 的两层并行结构, 可以通过调节进程数 *P* 与线程数 *T* 探索不同配比下的性能表现。

(四) MPI-IVF: 通信优化 (S5~S6)

从进程通信的角度来看, S1~S3 均为逐查询通信, 即每条查询都要执行一次集合通信。而当进程数较多且单查询计算量较小时, per-query 的通信启动开销将会成为不可忽视的一部分, 甚至会在延迟方面占据主导地位。S5 与 S6 则针对这一瓶颈进行了进程通信方面的优化。

S5 (批量通信) 先将所有查询先在本地全部搜索完, 将结果暂存于本地缓冲, 而后再用一次 MPI_Gather 统一收集全部查询的结果, 最后由 root 统一进行归并操作。通过这种方式, 可以将 2000 次小规模集合通信合并为单次大规模通信, 大幅减轻了进程间通信带来的开销。

Algorithm 3 MPI-IVF 批量通信 (S5)**Input:** 查询序列 $\{q_i\}_{i=0}^{M-1}$, k , $nprobe$, 进程号 $rank$, 进程总数 P **Output:** 全局 top- k 结果序列

```

1:                                     ▷ 本地全部搜索完 (无通信)
2: for  $i = 0$  to  $M - 1$  do
3:    $local\_q \leftarrow ivf\_search\_simd(q_i, nprobe, k)$ 
4:   将  $local\_q$  映射为全局 id 并拍平, 暂存至本地缓冲  $sbuf[i]$ 
5: end for
6:                                     ▷ 一次性进行 Gather (全部 query 合并, 单次大通信)
7:  $MPI\_Gather(sbuf) \rightarrow rbuf[]$                                      ▷  $P \times M \times k$  个候选汇集到 root
8:                                     ▷ root 统一归并
9: if  $rank = 0$  then
10:   for  $i = 0$  to  $M - 1$  do
11:      $result_i \leftarrow MergeTopK(\{rbuf, q_i\}, k)$ 
12:   end for
13: end if
14: return  $\{result_i\}$ 

```

S6 (非阻塞重叠) 采用 $MPI_Igatherv$ 配合双缓冲来实现计算与通信的流水线重叠。在搜索第 i 条查询的同时, 让第 $i - 1$ 条查询的结果通信在后台进行, 从而用流水线的方式隐藏了通信带来的延迟。

Algorithm 4 MPI-IVF 非阻塞重叠 (S6, 流水线)**Input:** 查询序列 $\{q_i\}$, 双缓冲 $buf \in \{0, 1\}$ **Output:** 全局 top- k 结果序列

```

1: 搜索  $q_0 \rightarrow buf_0$ ;  $MPI\_Igatherv(buf_0)$  发起通信
2: for  $i = 1$  to  $M - 1$  do
3:   搜索  $q_i \rightarrow buf_{i \bmod 2}$                                      ▷ 与上一条的通信重叠
4:    $MPI\_Wait$  等待  $q_{i-1}$  通信完成, root 归并  $q_{i-1}$  的结果
5:    $MPI\_Igatherv(buf_{i \bmod 2})$  发起  $q_i$  的通信
6: end for
7:  $MPI\_Wait$  并归并最后一条 query

```

(五) MPI-IVF-PQ (S7~S10)

MPI-IVF-PQ 将上述数据并行的思路迁移到 IVF-PQ 算法上, 各进程在自己的分片上先构建 IVF, 再建 PQ (考虑全局 PQ / 簇内独立 PQ 两种方法), 查询时调用上次实验中的 $ivf_pq_search_*$ / $ivf_pq2_search_*$ 来获取局部 top- k , 再复用与 IVF 完全一致的 Gather 进行归并。

具体实现上, S7/S9 使用 MPI 多进程并行, S8/S10 则在进程内叠加 OpenMP 与多线程结合。由于这部分的归并与 IVF 完全一致, 程序共用相同的逻辑, 这里我们直接复用前面所说的框架即可, 实现上较为简单。以 S7 为例:

Algorithm 5 MPI-IVF-PQ 方法一 (S7)

Input: 全量 base, 查询 q , k , $nprobe$, $rerank_k$, 进程号 $rank$, 进程总数 P

Output: 全局 top- k 结果 (仅 root 完整)

```

1:                                     ▷ 离线: 各进程在分片上 build_ivf, 再在分片上建全局 PQ
2:                                     ▷ 在线 (每条 query):
3:  $local\_q \leftarrow ivf\_pq\_search\_simd(q, nprobe, rerank\_k, k)$  ▷ ADC 粗排 + 精排, 局部下标
4: 将  $local\_q$  转换为  $(dist[], id[])$ , 并经  $shard\_ids$  映射 id 为全局
5:  $MPI\_Gather(dist) \rightarrow rd[]$ ;  $MPI\_Gather(id) \rightarrow rid[]$ 
6: if  $rank = 0$  then
7:    $result \leftarrow MergeTopK(rd, rid, k)$ 
8: end if
9: return  $result$ 

```

(六) MPI-HNSW (S11~S14)

在图索引 HNSW 这部分, 我们主要实现了两种不同的并行思路, 两者差异显著, 在并行逻辑上具有本质区别。

S11 (MPI-HNSW 多分块) 采用数据并行的思路。将数据按 block 切分, 每个进程在自己的分片上独立构建一张 HNSW, 查询时各自 `searchKnn` 并映射回全局 id 进行后续的 Gather 归并操作。由于各分片是数据的不同子集, 且各自独立进行搜索, P 的变化会同时影响最终结果的延迟与召回率。**S12** 在此基础上将每个分片进一步切成 T 个子图, 再在进程内使用 OpenMP 多线程进行子图搜索, 是多子图思路在 MPI 框架下的结合实现。

Algorithm 6 MPI-HNSW 多分块 (S11, 数据并行)

Input: 全量 base, 查询 q , k , 进程号 $rank$, 进程总数 P

Output: 全局 top- k 结果 (仅 root 完整)

```

1:                                     ▷ 离线: 按 block 取本进程分片  $shard\_ids$ , 在其上独立 build_hnsw
2:   (子图 label 为分片内局部下标, 故搜索结果需经  $shard\_ids$  还原为全局 id)
3:                                     ▷ 在线 (每条 query):
4:  $local\_q \leftarrow HNSW.searchKnn(q, k)$                                      ▷ 在本进程子图上贪心搜索
5: 将  $local\_q$  转换为  $(dist[], id[])$ , 并经  $shard\_ids$  映射 id 为全局
6:  $MPI\_Gather(dist) \rightarrow rd[]$ ;  $MPI\_Gather(id) \rightarrow rid[]$ 
7: if  $rank = 0$  then
8:    $result \leftarrow MergeTopK(rd, rid, k)$ 
9: end if
10: return  $result$ 

```

S13 (MPI-IVF-HNSW) 则采用任务并行的思路, 各进程都建立相同的全局 IVF, 并为每个 IVF 簇单独建立一张 HNSW, 簇按照 $c \bmod P$ 分配给各个进程持有。查询时各进程独立完成粗排 (该过程仅 `nlist` 个簇心点积, 开销极小), 选出 $nprobe$ 个候选簇, 然后对自己所持有的候选簇的 HNSW 进行搜索, 最后再 Gather 归并。由于簇内 HNSW 直接以全局 id 作为 label, 因此该策略在归并时无需再将其映射回原始 id。

这一策略的关键特征是, 无论进程数 P 取多少, 搜索得到的都是同样的候选簇集合, 因此其召回率与 P 无关, P 只会影响搜索的速度。这也是任务并行区别于数据并行的本质所在。

Algorithm 7 MPI-IVF-HNSW 任务并行 (S13)

Input: 查询 q , $nprobe$, 进程号 $rank$, 进程总数 P , 每簇 HNSW $\{H_c\}$

Output: 全局 top- k 结果

```

1:  $\triangleright$  离线: 全局 IVF 复制; 簇  $c$  由进程  $(c \bmod P)$  持有其 HNSW  $H_c$ 
2:  $\mathcal{P} \leftarrow$  粗排选  $nprobe$  个候选簇 (各进程独立计算, 结果一致)
3:  $local\_q \leftarrow \emptyset$ 
4: for  $c \in \mathcal{P}$  且  $c \bmod P = rank$  do  $\triangleright$  只搜本进程持有的候选簇
5:    $local\_q \leftarrow local\_q \cup H_c.searchKnn(q, k)$   $\triangleright$  label 即全局 id
6: end for
7: return Gather+Merge( $local\_q, k$ )

```

S14 (HNSW-on-HNSW) 采用两层图的结构, 与 S13 共用同一套簇内 HNSW, 两者唯一区别在于粗排阶段。S13 暴力扫描全部 $nlist$ 个簇心, 选取其中 $nprobe$ 个簇; 而 S14 则是额外在 $nlist$ 个簇心上构建一张顶层路由 HNSW, 用它来快速检索出 $nprobe$ 个最近的簇。即上层图负责路由, 下层图负责精搜。当 $nlist$ 很大时, 顶层 HNSW 具有对数级的复杂度, 相比暴力扫描簇心具有显著优势。

Algorithm 8 HNSW-on-HNSW 两层图查询 (S14)

Input: 查询 q , $nprobe$, 进程号 $rank$, 进程总数 P , 顶层路由图 H_{top} , 每簇 HNSW $\{H_c\}$

Output: 全局 top- k 结果

```

1:  $\triangleright$  离线: 全局 IVF 复制; 在  $nlist$  个簇心上构建顶层路由图  $H_{top}$ ; 簇  $c$  由进程  $(c \bmod P)$  持有  $H_c$ 
2:  $\mathcal{P} \leftarrow H_{top}.searchKnn(q, nprobe)$   $\triangleright$  用顶层的路由图选取簇, 替代暴力扫描簇心
3:  $local\_q \leftarrow \emptyset$ 
4: for  $c \in \mathcal{P}$  且  $c \bmod P = rank$  do  $\triangleright$  只搜本进程持有的候选簇
5:    $local\_q \leftarrow local\_q \cup H_c.searchKnn(q, k)$   $\triangleright$  下层图精搜, label 即全局 id
6: end for
7: return Gather+Merge( $local\_q, k$ )

```

(七) 批量测试框架

与上次实验一致, 由于策略与参数众多, 且服务器多人共享, 性能存在波动, 在 AI 的帮助下, 改写实现了一版命令行参数化的主程序 `r_main.cc` 以及对应的用于批量测试的脚本 `run_mpi_batch.py` (详见代码仓库)。脚本定义了测试矩阵, 对每个配置用 `mpirun -np P` 运行, 并进行多次测试, 取其中最优延迟作为结果, 再将最终的全部结果汇总输出。

四、实验结果分析

(一) 数据总览

本实验在 DEEP100K 数据集上对全部 15 个策略进行测试 (2000 次查询, $k = 10$)。由于服务器为多人共享, 性能存在波动, 我们对每个不同参数的配置进行多轮测量取延迟最小值作为结果, 并在凌晨人数较少时进行集中测试, 以尽可能减少性能波动带来的影响。本次以**无 MPI 的单进程串行 IVF (S0, $nprobe=16$, $380.3 \mu s$)** 作为全局基线 T_1 。表 1 对本次实验涉及各个策略具有代表性的测试结果进行了汇总。

表 1: 实验数据总览

策略	算法 / 并行方式	进程数 P	线程数 T	召回率	延迟 (μs)	加速比
S0	串行 IVF 基线 (无 MPI)	1	1	0.9583	380.3	1.00×
S1	MPI-IVF block + Gather	4	1	0.9629	133.2	2.86×
S2	MPI-IVF cyclic + Gather	4	1	0.9645	131.4	2.89×
S3	MPI-IVF block + 树形 Reduce	4	1	0.9629	145.0	2.62×
S4	MPI-IVF + OMP (2×2)	2	2	0.9611	129.2	2.94×
S5	MPI-IVF 批量通信	8	1	0.9680	63.1	6.03×
S6	MPI-IVF 非阻塞重叠	8	1	0.9680	76.4	4.98×
S7	MPI-IVF-PQ 方法一	4	1	0.9618	270.9	1.40×
S8	MPI-IVF-PQ 方法一 +OMP	2	4	0.9609	312.0	1.22×
S9	MPI-IVF-PQ 方法二	4	1	0.9628	670.9	0.57×
S10	MPI-IVF-PQ 方法二 +OMP	2	4	0.9611	396.0	0.96×
S11	MPI-HNSW 多分块	1	1	0.9810	402.3	0.95×
S12	MPI-HNSW + OMP	2	4	0.9988	424.4	0.90×
S13	MPI IVF+HNSW	4	1	0.9580	1181.5	0.32×
S14	HNSW-on-HNSW	4	1	0.9580	1232.2	0.31×

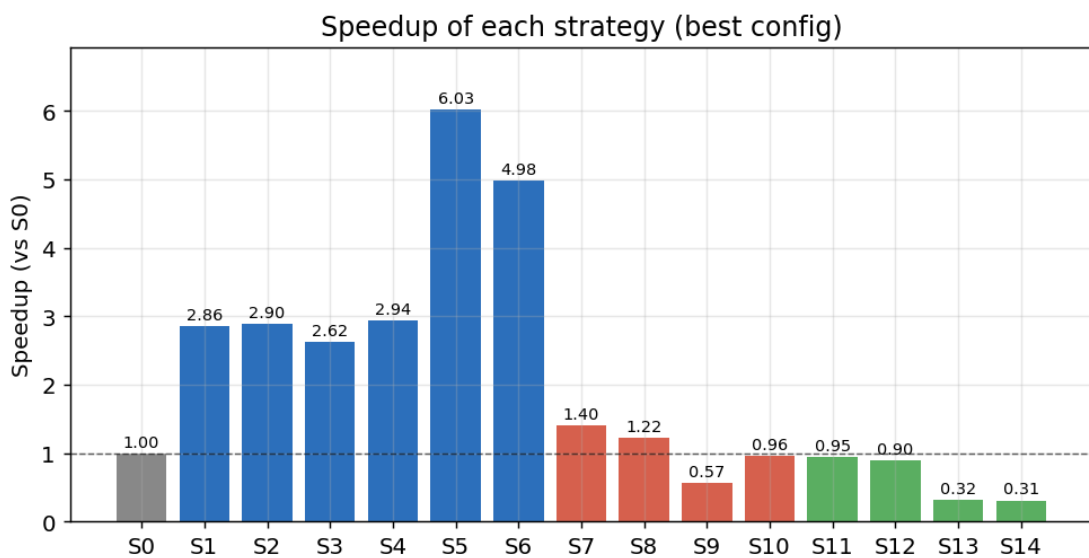


图 1: 各策略最优配置加速比对比 (相对 S0)

总体来看, 本实验中各个策略的优化效果差异较为明显。MPI-IVF 的通信优化效果最为显著, 批量通信 (S5) 在 $P = 8$ 下取得了相对 S0 **6.03×** 的加速比; 而 IVF-PQ 与图索引部分, 由

于算法本身单查询基线延迟较高，同时受通信与任务粒度限制，多进程的净收益相对有限，部分策略（S9/S11/S13/S14）甚至出现了**负优化**的情况。这种分化其实也揭示了 MPI 并行的收益边界，并非所有算法都能从多进程中获益，其优化效果高度依赖于算法的计算、访存与通信等特征。下面我们将对此展开深入的分析与讨论。

(二) 进程数扩展性分析

1. MPI-IVF 各通信策略随进程数的扩展

表 2 中记录了 MPI-IVF 四种通信策略在不同进程数下的延迟。

表 2: MPI-IVF 各通信策略随进程数的延迟 (μs)

策略	$P = 1$	$P = 2$	$P = 4$	$P = 8$	最优加速比
S1 逐查询 Gather	350.4	199.2	133.2	170.3	$2.86\times (P=4)$
S3 树形 Reduce	365.7	218.5	145.0	156.2	$2.62\times (P=4)$
S5 批量通信	364.3	189.5	98.2	63.1	$6.03\times (P=8)$
S6 非阻塞重叠	384.2	203.7	120.2	76.4	$4.98\times (P=8)$

从数据中可以归纳出几个清晰且互相印证的规律：

- **逐查询通信在 $P = 8$ 出现反噬**：S1 从 $P = 4$ 的 $133.2 \mu\text{s}$ 回升到 $P = 8$ 的 $170.3 \mu\text{s}$ 。原因在于， $P = 8$ 时单查询的本地计算量被切到仅约十余微秒，但每条 query 仍需一次 8 进程的集合通信、root 仍需线性归并 8×10 个候选，此时**通信与归并的固定开销已与计算本身相当甚至更高**，导致延迟不降反升。这是本实验关于“通信瓶颈”最直接的证据。
- **树形 Reduce 缓解了归并端的瓶颈**：S3 在 $P = 8$ 下为 $156.2 \mu\text{s}$ ，反超了同等条件的 S1 ($170.3 \mu\text{s}$)。这是因为 Gather 需把全部候选汇集到 root 做 $O(P \cdot k)$ 的线性归并，而 Reduce 以 $O(\log P)$ 的树形深度完成、归并计算分摊到各层进程， P 越大其相对优势越明显——在 $P \leq 4$ 时 S3 还略慢于 S1（多了规约算子的调用开销），到 $P = 8$ 时则实现反超。
- **批量通信效果最佳**：S5 在 $P = 8$ 下仅 $63.1 \mu\text{s}$ ，相对 S1 的 $170.3 \mu\text{s}$ 快约 $2.7\times$ ，且其延迟随 P **持续单调下降** ($364 \rightarrow 190 \rightarrow 98 \rightarrow 63$)，完全没有出现反噬。将 2000 次小规模集合通信合并为 1 次大规模通信，彻底摊掉了 per-query 的通信启动开销，是应对逐查询通信瓶颈最有效的手段。
- **非阻塞重叠有真实收益**：S6 在 $P = 8$ 下为 $76.4 \mu\text{s}$ ，介于树形 Reduce 与批量通信之间，说明该 MPI 环境下 MPI_Igather 确实存在一定的异步推进能力，计算与通信得到了部分重叠，但其效果不及“从根本上减少通信次数”的批量通信彻底。

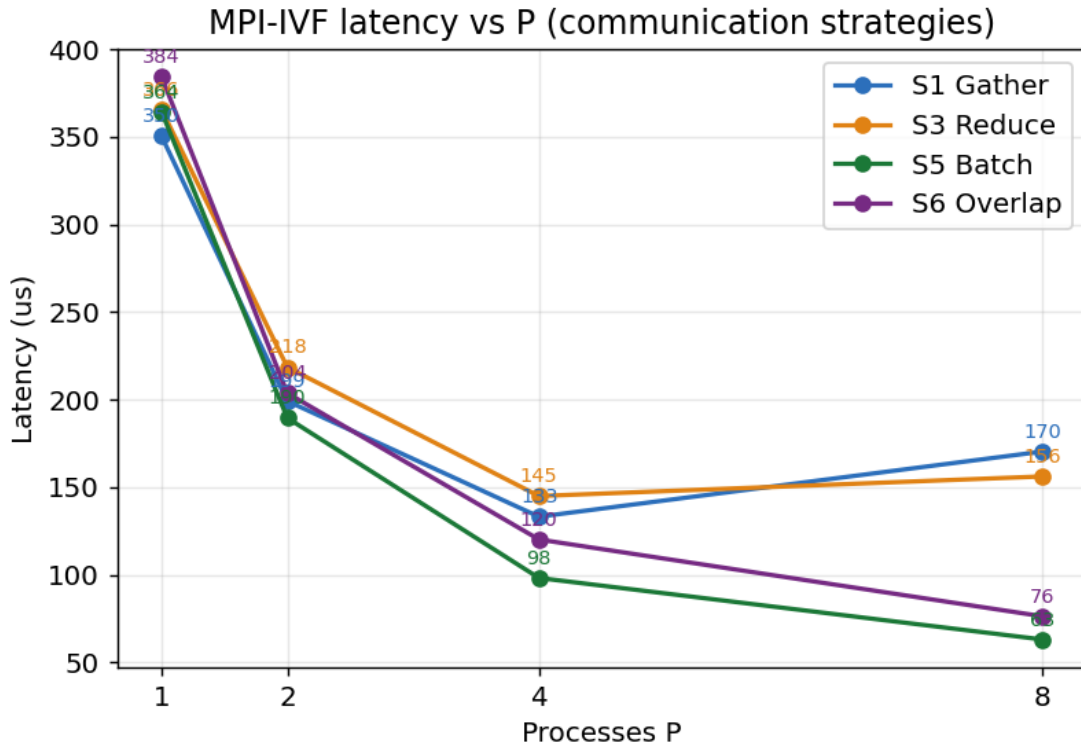


图 2: MPI-IVF 不同通信策略的扩展性折线图

2. 进程数对召回率的影响

观察数据我们还能发现, MPI-IVF 的召回率也会随 P 而上升 ($P = 1$ 的 0.9583 升至 $P = 8$ 的 0.9680)。这与多线程实验中的 HNSW 多子图并行中, 用广度换取召回率是同一原理, 即每个分片各自扫描 $nprobe$ 个簇, P 个分片的实际效果其实等效为扫描了更大范围的候选区域, 从而使得整体的召回率有所提升。这意味着在数据并行的框架下, 增加进程数不仅能降低延迟, 往往还能额外带来召回率的提升。

(三) 任务划分方式对比

表 3 对比了块划分 (S1) 与循环划分 (S2) 两种任务划分方式的测试结果。从数据来看, 两者延迟较为接近, 各分片数据量较为均匀, 差异主要体现在召回率上, 即循环划分的召回率总是略高于块划分。

表 3: block 与 cyclic 划分对比 (延迟 μs / 召回率)

划分方式	$P = 2$	$P = 4$	$P = 8$
S1 block	199.2 / 0.9611	133.2 / 0.9629	170.3 / 0.9680
S2 cyclic	198.9 / 0.9618	131.4 / 0.9645	241.9 / 0.9688

虽然召回率差异较小, 但是这种趋势却很稳定, 其原因主要在于数据分布的局部性质。块划分让连续 id 的向量进入同一进程, 而原始数据中相邻 id 的向量往往具有一定的局部聚集性, 这

使得各个分片的数据分布出现了一定的偏差；循环划分则将向量打散到各个进程，让每个分片的数据分布更接近全局，分布更加均匀，各分片独立 KMeans 得到的簇也更具代表性，从而在归并后获得略高的召回。这一差异虽小，但印证了划分方式不仅影响负载，也会通过改变各分片的数据分布，间接对搜索的精度造成影响。

(四) MPI 与多线程混合并行 ($P \times T$)

表 4 统计了 S4 在不同 $P \times T$ 配比下的延迟，并与同等总核数的纯 MPI 策略 (S1) 进行了对照。

表 4: MPI×OpenMP 不同配比延迟 (μs)

配比 ($P \times T$)	1×1	1×2	1×4	1×8	2×2	2×4	4×2
S4 延迟	391.5	200.4	137.7	165.6	129.2	133.6	134.8
S1 ($T=1$)	$P=2$: 199.2 $P=4$: 133.2 $P=8$: 170.3						

分析表中数据可知，总核数固定时存在最优的配比。在 $P \cdot T = 8$ 的几种组合中， 2×2^1 (129.2 μs) 与 2×4 (133.6 μs)、 4×2 (134.8 μs) 均优于纯线程的 1×8 (165.6 μs)。这说明适度地将多进程与多线程相结合，可以取得很好的优化效果。纯线程 (1×8) 受限于单进程内簇级 OpenMP 的任务粒度，纯进程 (S1 的 $P = 8$) 又受限于逐查询通信开销，二者结合反而取得了平衡。

同时，混合的并行策略还能缓解纯 MPI 的通信开销。S1 在 $P = 8$ 时因逐查询带来的通信开销，延迟升至 170.3 μs ，而 S4 使用 2×4 ，同为 8 核，延迟却只有 133.6 μs 。也就是说，使用更少的进程（更少的通信）配合进程内的多线程并行，可以有效避开进程数 P 过多时的通信瓶颈。此外，当 $P \cdot T$ 超过物理核数时，也会出现超额订阅导致的严重负优化。经过测试发现，其负优化相当严重且不稳定，这里我们不做讨论；本实验仅考虑混合配比在 $P \cdot T \leq 8$ 的正常场景下的数据。

¹严格来说， 2×2 仅占用 4 核，但其延迟已是全表最低； $P \cdot T = 8$ 的组合中 2×4 (133.6) 与 4×2 (134.8) 接近，但都略慢于 2×2 。

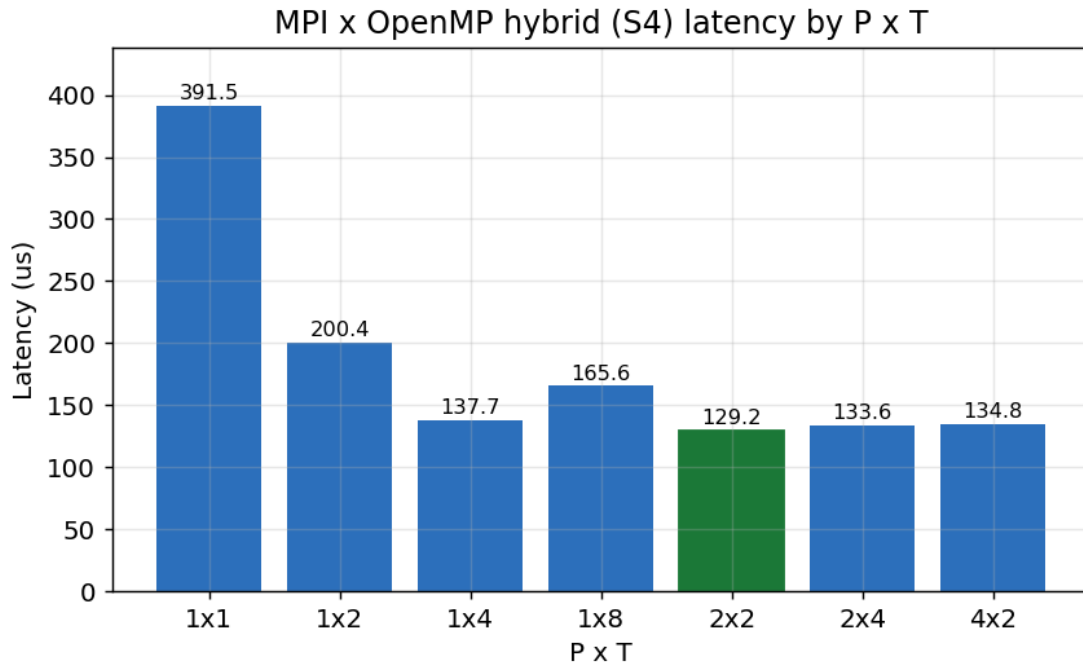


图 3: MPI×OpenMP 混合并行配对比

(五) local_topk 的影响

在 MPI 的归并过程中，每个进程分片返回的候选数 `local_topk` 也会影响通信量与归并质量，为结果带来一定的影响。表 5 统计了 S1 在不同 `local_topk` 下的延迟。

表 5: `local_topk` 对延迟的影响 (S1, 延迟 μs)

	<code>local_topk=10</code>	<code>local_topk=20</code>	<code>local_topk=50</code>
$P = 4$	133.2	140.5	173.5
$P = 8$	170.3	193.8	200.3

由于本实验数据并行下各分片召回数已经较大 ($k = 10$)，继续增大 `local_topk` 并不能带来召回率的提升，反而增加了 Gather 的通信开销与 root 的归并负担，导致其延迟随之有所上升 ($P = 8$ 时从 170 升至 200 μs)。因此在后续测试中，统一取 `local_topk = k = 10`，在保证召回的同时，使通信带来的开销尽量减少。

(六) 参数权衡分析

前面我们已分别考察了进程数 P 、`local_topk` 等单一维度的影响。然而在实际的 MPI 并行中，这些参数并非彼此独立，而是共同决定着最终的性能与召回精度。这里我们从多参数的角度，综合各个参数之间的相互关系，对其进行权衡分析。

1. 进程数 P 与通信策略

通过实验我们知道，通信策略决定了 P 的可扩展上限，而 P 又决定了不同通信策略之间的差距。这一关系在表 2 中有明显的体现，在 $P \leq 4$ 时，四种通信策略的延迟差异并不大（ $P = 4$ 时分别为 133/145/98/120 μs ），因为此时单查询的计算量仍占主导，通信占比小，使用哪种通信几乎无所谓；但当到了 $P = 8$ ，计算粒度减小、通信开销变得明显，策略之间的差距骤然拉大（170 vs 156 vs 63 vs 76 μs ），逐查询 Gather 甚至出现不降反升的结果。

虽然我们已经知道这一现象的原因，但是从工程视角来看，这也意味着 P 越大，通信策略的选择就越关键。在当今的互联网大数据以及科学计算等工程化的场景中，节点数常常达到几十上百；对于大规模的模拟仿真甚至可以达到上千的总进程数。在这样的场景中，通信策略带来的影响往往是一切的关键。当 P 较小时，我们可以随意选择最简单的逐查询 Gather，不同通信策略间的差别并不明显，甚至也有可能会出现更优的策略反而在较小并行规模下表现更差的结果；而当希望通过多节点增大 P 来进一步提速时，就必须同步进行通信策略的优化。换言之，多进程与通信优化往往需要配套进行，对于其中任意一方进行单独的调整，都会很快触及性能的瓶颈。

2. 进程数 P 与算法参数 (nprobe / ef)

P 与算法自身的参数也存在一定联系，且这种相互作用会同时影响召回率与延迟两个方面。以 IVF 为例，单查询的计算量正比于 nprobe，而 MPI 将这部分计算切分到 P 个进程。因此当 nprobe 较大时（如 64），单查询计算量充足，增大 P 的收益显著，且进程间通信的占比很少；而当 nprobe 较小时（如 4），单查询计算本就只有几十微秒，此时增大 P 则很可能使通信开销成为主导，导致负优化。也就是说，nprobe 越小， P 的有效扩展范围也就越小，二者的选取也需综合考虑。

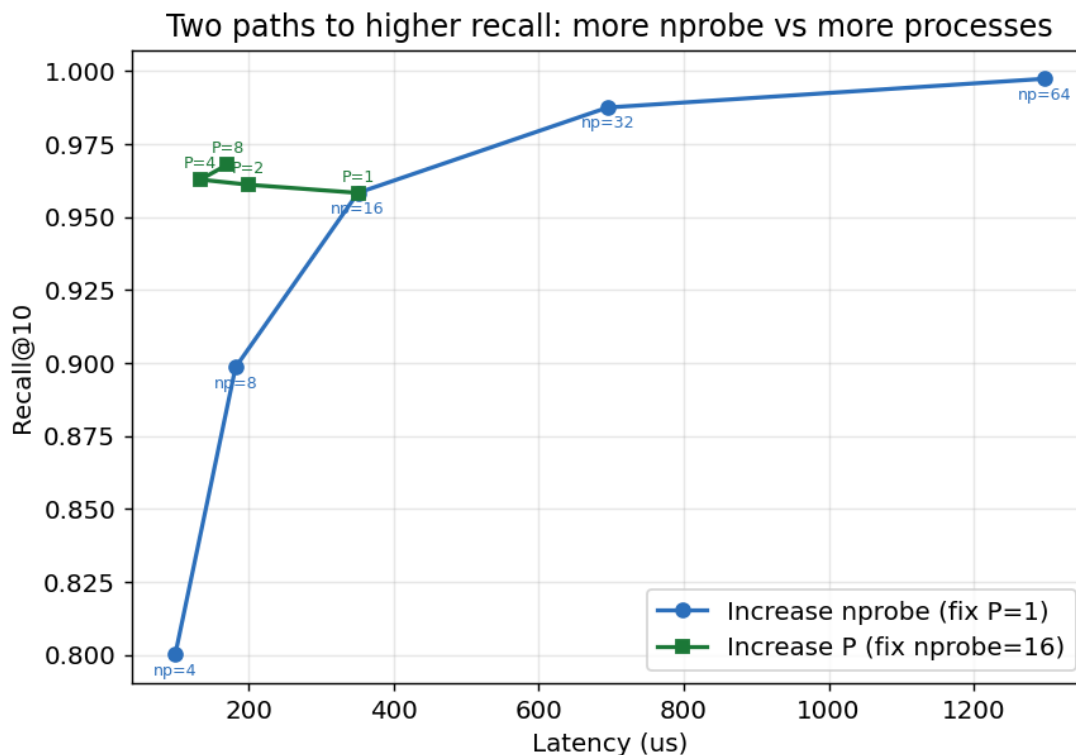


图 4: 增大 nprobe 与增大进程数 P 两条提升召回的路径对比

在召回层面，这一关系则更为复杂。数据并行下每个分片各自扫描 $nprobe$ 个簇， P 个分片联合的等效搜索范围约为 $P \times nprobe$ 。因此增大 P 与增大 $nprobe$ 对召回有相似的提升作用，带来了权衡的空间。如果目标是更高精度的召回，既可以增大单进程的 $nprobe$ （延迟线性上升），也可以增大进程数 P （在通信开销较低的前提下延迟会下降）。这一现象其实在 MPI-HNSW 多分块中也有类似规律，单图的 ef 与进程数 P 共同决定召回， P 个进程各用较小的 ef ，联合覆盖更大范围，可以在相近召回下取得比单进程较大 ef 更低的延迟。这说明在并行框架下，算法参数与并行参数可以相互替代或产生叠加的影响，调参时不应孤立地进行调整，通过参数扫描进行反复实验，往往能得到一个最优的组合。

3. 综合权衡

综合上述关系，可以发现 MPI 并行的调优是一个多参数共同影响的复杂过程，需要根据优化目标来多方面选择。结合本次实验的数据，可以归纳出几条面向不同目标的参数组合建议：

- **追求计算速度**：优先使用批量通信与较大的进程数，并配合较小的 $local_topk (=k)$ 以减少通信量；
- **逐查询场景**：选择适中的进程数 + 树形 Reduce，避免进程数过高带来的通信反噬；
- **追求更优召回**：可以考虑增大 $nprobe/ef$ 与增大进程数之间权衡，在通信开销占主导前优先选择增大进程数来进行优化；
- **单机核数受限或访存密集任务**：采用 MPI×OpenMP 混合的适中配比，使用进程内多线程，在保持并行度的同时减少通信带来的开销。

这些组合的核心思想是一致的，在 MPI 框架下，任何单一参数的最优配置都依赖于其它参数的取值，只有将进程数、通信策略、算法参数、混合配比多方面进行综合考虑，才能真正探索出性能的上限所在。这也是 MPI 多进程并行在调优上更复杂，同时也更具工程价值的地方。

(七) 数据并行与任务并行对比

通过前面的原理讲解，我们可以知道，图索引 HNSW 部分的两种并行思路具有本质的差异，在表 6 中统计了该组策略在不同进程数下的测量结果：

表 6: 图索引策略随进程数的延迟 (μs) 与召回率

策略	$P = 1$	$P = 2$	$P = 4$	$P = 8$	召回率变化趋势
S11 MPI-HNSW 多分块（数据并行）	402.3	484.0	601.2	803.2	0.981→0.999（随 P 提升）
S13 IVF+HNSW（任务并行）	1986.7	1377.4	1181.5	1327.3	0.9580（恒定）
S14 HNSW-on-HNSW（任务并行）	2031.8	1382.1	1232.2	1434.2	0.9580（恒定）

通过对比可以看出两种并行思路的本质区别，反映在数据上，表现为不同进程数带来的结果差异：

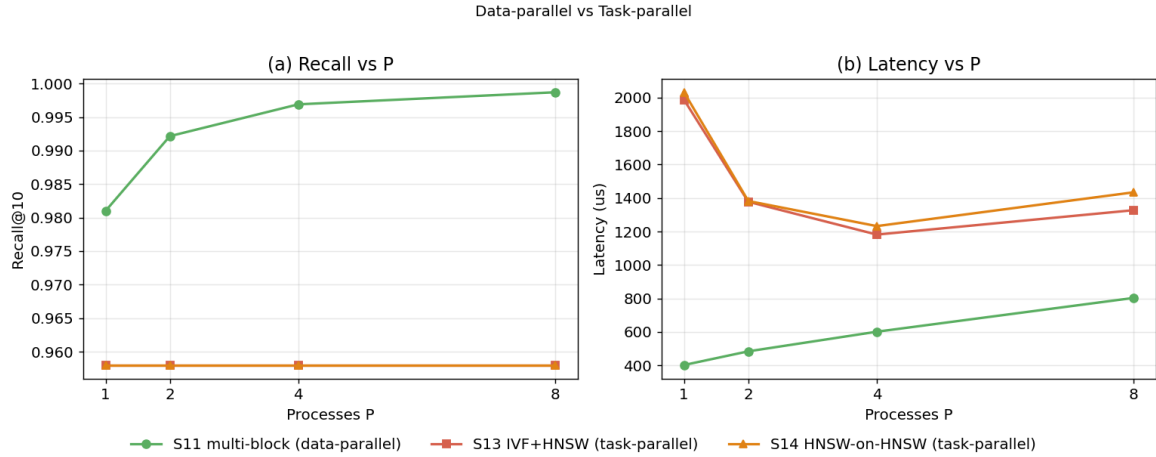


图 5: 数据并行 (S11) 与任务并行 (S13/S14) 召回率与延迟随 P 的对比

S11 (数据并行): P 改变结果, 但在本场景下出现了负优化 数据被切分成不同子集、各分片各自近似搜索, 因此召回率会随 P 上升 ($P = 1$ 的 0.981 升至 $P = 8$ 的 0.999)。然而其延迟却随 P 不断增大 (402→484→601→803 μ s), 这呈现出了明显的负优化。经过多次测试该趋势保持一致, 并非偶然抖动造成的。通过分析可知, 其根源其实在于, HNSW 单图搜索本身已经很快 (数百微秒量级) 且属于访存密集型任务, 将数据切成 P 份后, 每个子图虽小, 但其单次 `searchKnn` 的固定开销 (入口下降、维护 `ef` 大小的候选集等) 并不会等比例缩小, 而且为了达到 `ef=64` 的搜索宽度, 在小图上往往需要遍历更大比例的节点; 再加上逐查询 `Gather` 的带来的通信开销, 最终使得多分图的收益被”固定开销 + 通信”完全抵消甚至达到了负优化的效果。这也说明数据并行并非对所有算法都有效, 对于单查询本就很快的图索引, 在本次实验的数据规模下进行多分块完全得不偿失, 是一个典型的负优化案例。

S13/S14 (任务并行): P 不改变结果 无论 P 取多少, 粗排选出的都是同样的 `nprobe` 个候选簇, 搜索的也是同样是这批簇的同一组 HNSW, 只是把”搜哪些簇”这一任务按 $c \bmod P$ 分给了不同进程来并行执行, 最终归并结果与单进程完全一致, 因此召回率恒定不变与 P 严格无关。其延迟随 P 先降后微升 ($P = 1$ 的 1987 降至 $P = 4$ 的 1182, $P = 8$ 微升至 1327) 则属正常现象。 $P \leq 4$ 时簇间并行有效分摊了搜索负载, 而当 $P = 8$ 时 16 个候选簇分给 8 进程, 每个进程分配到的仅约 2 簇, 任务粒度过细, 通信占比上升, 于是出现了轻微的回升。这其实是任务并行最典型的扩展特征。

S14 与 S13 的对比 在 `nlist=256` 时, 顶层路由 HNSW 选簇与暴力扫描 256 个簇心的速度基本相当, 且选出的候选簇也一致 (召回率相同)。顶层路由结构要在 `nlist` 很大时才能体现其对数级检索的优势, 这一点本实验受数据规模所限未能充分展现, 但其结构本身是完整而有效的。

此外, 我们还能观察到 S13/S14 的整体延迟均显著高于 S11。这是因为在该策略下, 每条 query 都需对 `nprobe` 个候选簇分别调用 `searchKnn`, 即多次小图搜索及各自的入口建立, 其开销远大于 S11 的一张图搜一次, 这其实是 IVF + HNSW 这类结构自身决定的。而 S12 (多分块 + 进程内 OpenMP) 在 $P = 2$ 、 $T = 4$ 下可以达到 0.9988 的超高召回率与 424 μ s 的延迟, 是图索引这部分中综合表现较好的配置, 这也体现出多进程与多线程的结合策略带来的稳定收益。

(八) 问题探讨

在本次实验中，也出现了许多值得详细探讨的问题。

1. 为什么逐查询通信在进程数增多时反而负优化？

这是本实验最核心的现象。在前面我们虽然已经有所提及，但是还可以从更加深入的角度进行探讨。MPI 的集合通信（如 MPI_Gather）本身存在固定的启动与同步开销，且与进程数正相关。当 P 较小时，单查询的本地计算量仍占主导，通信开销可被掩盖；但当 $P = 8$ 、单查询本地计算被切到仅约十余微秒时，每条 query 的一次集合通信 + root 的线性归并已经与计算量处于同一数量级甚至更高，于是延迟不降反升。

针对这一问题，优化方向主要有三种思路：**改变归并的算法复杂度**（树形 Reduce, $O(P \cdot k) \rightarrow O(\log P)$ ）、**改变通信的粒度**（批量通信， M 次小通信 $\rightarrow$ 1 次大通信）、以及**隐藏通信延迟**（非阻塞重叠）。这三种策略在本实验中均得到验证，其中批量通信的效果最为彻底。这与多线程实验中线程创建/同步开销在任务粒度过小时占主导的现象（即线程池的相关问题）在思想上是相通的，都是固定开销在细粒度场景下的主导效应。

而这类问题的解决思路也较为类似，上述三点其实都是较为通用的优化方向，通过这几次实验中类似的过程也都得到了有效的验证。

2. 为什么 IVF-PQ 与图索引的 MPI 化净收益有限甚至负优化？

观察表 1，S7（方法一） $P = 4$ 仅 $1.40\times$ 加速，S9（方法二）与 S11/S13/S14 甚至低于基线。其原因可归为两类：其一是**算法本身基线延迟的差异**，IVF-PQ 多了 LUT 构建、ADC 扫描、精排这三个阶段，而 IVF+HNSW 则每查询要搜多个候选簇，二者的单查询基线本就远高于纯 IVF，而 S0 基线（纯 IVF， $380 \mu s$ ）已经具有显著的效率优势，使得这些流程更为繁重的算法即使并行后也难以超越基线。其二则是**访存与通信瓶颈**，IVF-PQ 属于访存密集型（多线程实验中的 perf 分析已经分析了其对内存带宽的依赖），多进程同时访问各自的 LUT 与编码表时带宽争用加剧；图索引则受单查询固定开销与逐查询通信的双重制约。这与上次实验中“IVF-PQ 多线程边际收益最低”的结论一致，无论多线程还是多进程，访存密集型算法的并行扩展性都受限内存带宽这一共享资源。这其实也提醒我们，并行优化必须结合算法的自身特征，盲目增加进程数未必有效。

3. MPI 与多线程的取舍与互补

通过本次多进程实验与上次多线程实验的对照可以看出，MPI 与多线程并非互斥，而是分层互补的。多线程在共享内存内规避了通信开销，更适合单节点、细粒度的并行优化；MPI 则能跨越进程乃至节点的内存边界，更适合粗粒度、多机节点的并行，但代价是进程间的通信开销。混合并行（S4）的结果表明，二者结合往往能够取长补短，用适度的进程数实现跨内存边界的扩展，同时用进程内多线程减少通信开销，从而规避纯 MPI 在高进程数下的通信反噬（S4 的 2×4 优于 S1 的 $P = 8$ ）。在实际工程中，也应根据问题规模、硬件条件（单机/多机、核数）与算法的计算/访存/通信特征等方面，全面而综合地考虑并行策略与配比问题。

五、 总结与体会

在本次实验中，我们在 lab2 SIMD 与 lab3 多线程的基础上，进一步将 ANN 搜索迁移到 MPI 多进程并行，系统地实现并对比了任务划分、通信归并、混合并行三个层面的多种策略，完成了 S0~S14 共 15 个策略的实现测试与分析。

从实验结果来看，MPI 并行的收益与瓶颈都高度依赖于通信开销的占比。MPI-IVF 的批量通信在 $P = 8$ 下取得了相对串行 IVF 基线 $6.03\times$ 的加速，而同样算法的逐查询 Gather 在 $P = 8$

时却因通信反噬出现了负优化。同一算法、不同通信策略，性能可以相差数倍。这让我深刻体会到，在 MPI 编程中，通信往往比计算更能决定最终性能。树形 Reduce、批量通信、非阻塞重叠这三种优化，分别从归并复杂度、通信粒度、通信隐藏三个角度有效缓解了瓶颈，也是本实验中很有价值的收获。

另一个体会则来自于数据并行与任务并行的对比。图索引部分，MPI-HNSW 多分块（数据并行）的召回率随进程数上升，而 IVF+HNSW（任务并行）的召回率与进程数严格无关。前者把数据切开各自近似，后者把任务进行切分，协同完成同一件事。这组对照让我理解了两种并行思路的本质差异，以及它们对结果正确性与扩展性的不同影响。

总体而言，本次实验完整地实现了 MPI 多进程编程的内容，涵盖了任务划分、通信归并、流水线与混合并行等核心问题。结合前两次实验，我对于“指令级—线程级—进程级”这一完整的并行层次有了更加系统而深入的认识，也为今后在实际工程中针对不同硬件条件与算法特征选择合适的并行策略积累了宝贵经验。

MINIB

六、 附录: 代码仓库链接

GitHub: <https://github.com/Iriasoul/Parallel-Programming/tree/main/lab04>

NIKU